



Федеральное государственное автономное образовательное
учреждение высшего образования
«Национальный исследовательский университет ИТМО»

ФАКУЛЬТЕТ «Программной инженерии и компьютерной техники»

ОБРАЗОВАТЕЛЬНАЯ ПРОГРАММА «Проектирование и разработка систем больших данных»

НАПРАВЛЕНИЕ ПОДГОТОВКИ (СПЕЦИАЛЬНОСТЬ) «09.04.04 Программная инженерия»

ОТЧЕТ

по лабораторной работе №4

по курсу «Структуры и алгоритмы в базах данных и распределенных
системах»

на тему: «Реализация конкурентной хэш-таблицы»

Обучающийся: Постнов С. А., Р4135
(Ф.И.О, № группы)

Преподаватель Платонов А. В., доцент
(Ф.И.О, должность)

Преподаватель Портнов П. В., ассистент
(Ф.И.О, должность)

2026 г.

СОДЕРЖАНИЕ

1	Теоретическая часть	3
1.1	Конкурентные хэш-таблицы	3
1.2	Striped locking	3
1.3	Хэш-функция FNV-1a	4
2	Практическая часть	5
2.1	Архитектура реализации	5
2.2	Базовая реализация	5
3	Исследовательская часть	6
3.1	Аппаратные характеристики	6
3.2	Методика замеров	6
3.3	Результаты для одного потока	7
3.4	Результаты для нескольких потоков	9
3.5	Потребление памяти	12
3.6	Профилирование	14
4	Вывод	18

1 Теоретическая часть

1.1 Конкурентные хэш-таблицы

Конкурентная хэш-таблица — структура данных, допускающая одновременный доступ из нескольких потоков выполнения без нарушения целостности данных. В отличие от обычной хэш-таблицы, где для синхронизации можно использовать единственную блокировку (*coarse-grained locking*), конкурентные реализации стремятся минимизировать конкуренцию за блокировки, чтобы обеспечить масштабируемость при увеличении числа потоков.

Основные подходы к реализации конкурентных хэш-таблиц:

- *Striped locking* — множество независимых блокировок, каждая из которых защищает свою полосу (группу бакетов). Количество блокировок фиксировано и не зависит от размера таблицы, что упрощает реализацию, но может приводить к ложной конкуренции при ресайзе;
- *Lock striping с переносимыми блокировками* — аналог striped locking, но при ресайзе блокировки переносятся вместе с бакетами, что позволяет избежать глобальной блокировки;
- *Lock-free реализации* — используют атомарные операции (CAS) для обновления бакетов без блокировок. Обеспечивают максимальную масштабируемость, но сложны в реализации, особенно при ресайзе;
- `sync.Map` (Go) — встроенная конкурентная карта Go, использующая двухуровневую схему: `read-only` кэш для быстрого чтения и `dirty`-карта для записи с амортизированным переносом.

1.2 Striped locking

В данной работе используется подход *striped locking*: каждый бакет таблицы защищён собственным `sync.RWMutex`. Это позволяет нескольким потокам одновременно читать из разных бакетов, а запись блокирует только один бакет. Для ресайза используется отдельная блокировка `resizeMu`: при добавлении элемента поток захватывает `resizeMu.RLock()`, что позволяет нескольким потокам одновременно выполнять `Put`, а при необходимости

ресайза — `resizeMu.Lock()`, блокируя все операции добавления на время перестройки таблицы.

Текущая таблица хранится в `atomic.Value`, что обеспечивает lock-free доступ к указателю на таблицу при чтении. При ресайзе создаётся новая таблица увеличенного размера, после чего указатель атомарно заменяется.

1.3 Хэш-функция FNV-1a

FNV-1a (Fowler–Noll–Vo) — некриптографическая хэш-функция, хорошо подходящая для хэш-таблиц благодаря быстрому вычислению и равномерному распределению. Алгоритм: для каждого байта входных данных выполняется операция XOR с текущим хэшем, затем результат умножается на простое число (prime). Начальное значение хэша (offset basis) и множитель выбраны эмпирически для минимизации коллизий.

2 Практическая часть

2.1 Архитектура реализации

Основной код расположен в файле `concurrent/map.go`. Структура данных состоит из следующих компонентов:

- 1) `Map` — основная структура, содержащая:
 - `resizeMu sync.RWMutex` — блокировка ресайза (чтение при `Put`, запись при `maybeResize`);
 - `table atomic.Value` — указатель на текущую таблицу (lock-free чтение);
 - `size int64` — счётчик элементов (атомарный);
 - `maxLoadFactor float64` — порог загрузки (по умолчанию 0.75).
- 2) `table` — контейнер для массива бакетов, хранящийся в `atomic.Value`;
- 3) `bucket` — бакет с `sync.RWMutex` и слайсом `[]entry`;
- 4) `entry` — пара ключ-значение (`key string, value string`).

2.2 Базовая реализация

Файл `concurrent/map_baseline.go` содержит простую обёртку над `map[string]string` без синхронизации, используемую как базовый вариант для сравнения в однопоточных бенчмарках. Для многопоточного сравнения используется стандартная `sync.Map` (обёртка `MapSync` в `benchmark_test.go`).

3 Исследовательская часть

3.1 Аппаратные характеристики

Все замеры проводились на ноутбуке со следующей конфигурацией:

- операционная система: macOS 26.3.1 (arm64);
- процессор: Apple M3 Pro, 11 ядер;
- оперативная память: 18 ГБ;
- версия Go: go1.25.

Во время замеров не выполнялись ресурсоёмкие фоновые задачи, а ноутбук был подключён к сети электропитания для обеспечения максимальной производительности.

3.2 Методика замеров

Для замеров использованы встроенные benchmark-тесты Go (`testing.B` с `b.Loop()`). Измерялись две операции: `put` и `get` в двух режимах:

- *single* — однопоточное выполнение (сравнение с `MapBaseline`);
- *multi* — многопоточное выполнение с числом потоков, равным `GOMAXPROCS` (сравнение с `sync.Map`).

Для каждого размера набора данных ($N = 1000, 2500, 5000, 7500, 10000, 25000, 50000, 100000$) фиксировались:

- средняя задержка на операцию (нс/оп);
- средняя пропускная способность (оп/с);
- 95% доверительный интервал (CI95) для задержки и пропускной способности;
- потребление кучи на операцию (байт/оп).

Доверительный интервал вычисляется как $CI_{95} = 1.96 \cdot \sigma / \sqrt{n}$, где σ — стандартное отклонение выборки, n — число итераций. Потребление памяти измеряется через `runtime.ReadMemStats` как разница `HeapAlloc` до и после выполнения пакета операций.

Для профилирования CPU используется отдельная программа `cmd/profile/main.go`, которая выполняет заданное число раундов операций с включённым `pprof.StartCPUProfile`. Профили визуализируются как ориентированные графы вызовов с помощью `go tool pprof -dot` и `dot -Tpdf`.

3.3 Результаты для одного потока

Таблица 3.1 содержит результаты замеров задержки в однопоточном режиме.

Таблица 3.1 – Задержка операций в однопоточном режиме (нс/оп)

N	вставка (concurrent)	вставка (baseline)	поиск (concurrent)	поиск (baseline)
1000	97.10 ± 0.16	45.31 ± 0.04	14.31 ± 0.01	10.14 ± 0.00
2500	89.68 ± 0.08	42.45 ± 0.04	14.71 ± 0.04	10.90 ± 0.01
5000	88.69 ± 0.08	39.54 ± 0.03	14.04 ± 0.01	11.10 ± 0.01
7500	107.09 ± 0.11	45.48 ± 0.05	14.46 ± 0.01	10.56 ± 0.01
10000	94.41 ± 0.69	38.53 ± 0.03	14.34 ± 0.01	10.63 ± 0.01
25000	216.08 ± 1.44	55.14 ± 0.12	16.68 ± 0.10	11.51 ± 0.01
50000	189.63 ± 1.31	52.66 ± 0.45	19.50 ± 0.17	11.59 ± 0.01
100000	203.46 ± 3.42	58.76 ± 0.39	33.55 ± 0.24	12.00 ± 0.04

Таблица 3.2 содержит результаты замеров пропускной способности в однопоточном режиме.

Таблица 3.2 – Пропускная способность в однопоточном режиме (млн оп/с)

N	вставка (concurrent)	вставка (baseline)	поиск (concurrent)	поиск (baseline)
1000	10.36 ± 0.01	22.19 ± 0.02	70.27 ± 0.03	99.10 ± 0.04
2500	11.16 ± 0.01	23.60 ± 0.02	68.74 ± 0.07	92.14 ± 0.06
5000	11.28 ± 0.01	25.31 ± 0.02	71.47 ± 0.06	90.63 ± 0.08
7500	9.34 ± 0.01	22.01 ± 0.03	69.33 ± 0.06	94.96 ± 0.08
10000	10.67 ± 0.04	25.97 ± 0.02	69.84 ± 0.06	94.27 ± 0.08
25000	4.64 ± 0.03	18.15 ± 0.04	61.13 ± 0.29	86.95 ± 0.07
50000	5.28 ± 0.03	19.14 ± 0.15	52.12 ± 0.33	86.30 ± 0.07
100000	4.93 ± 0.08	17.06 ± 0.11	29.92 ± 0.17	83.48 ± 0.16

Графики результатов представлены на рисунках 3.1 и 3.2.

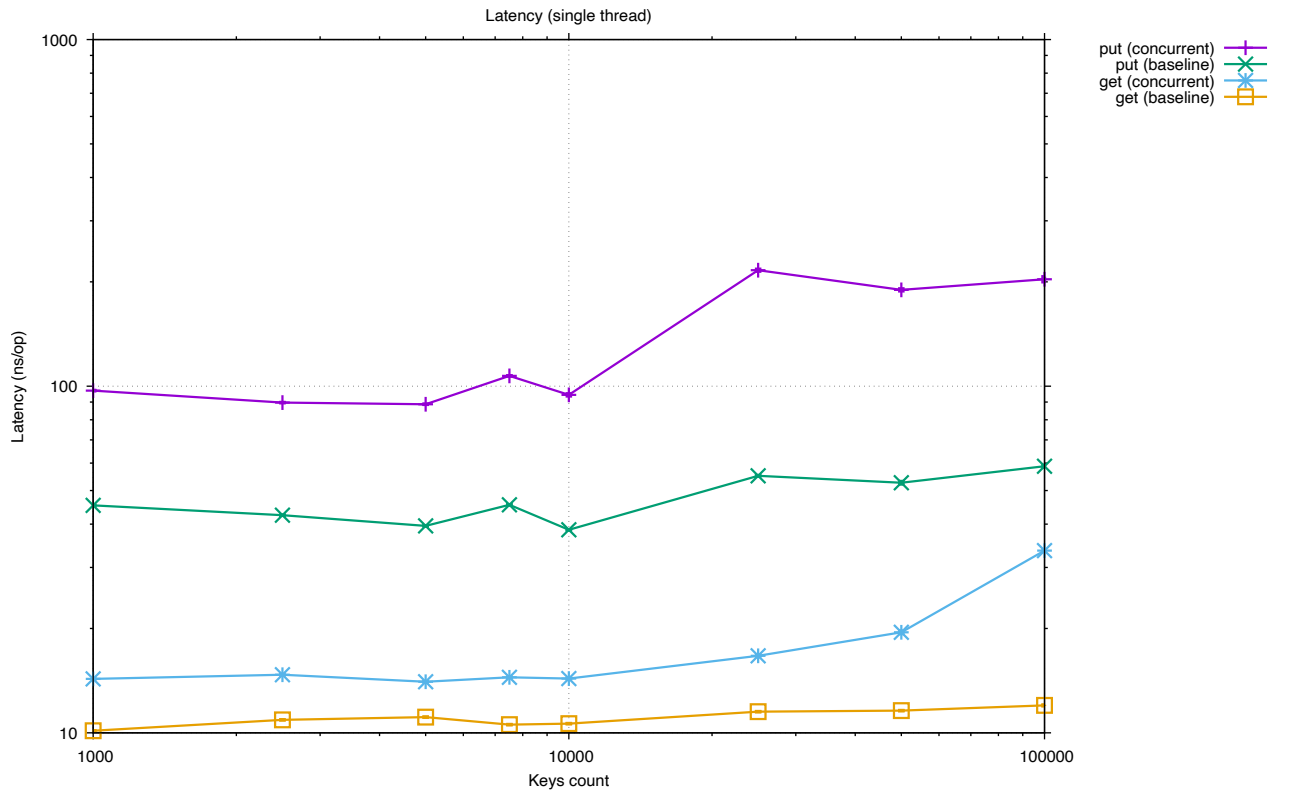


Рисунок 3.1 – Задержка операций в однопоточном режиме

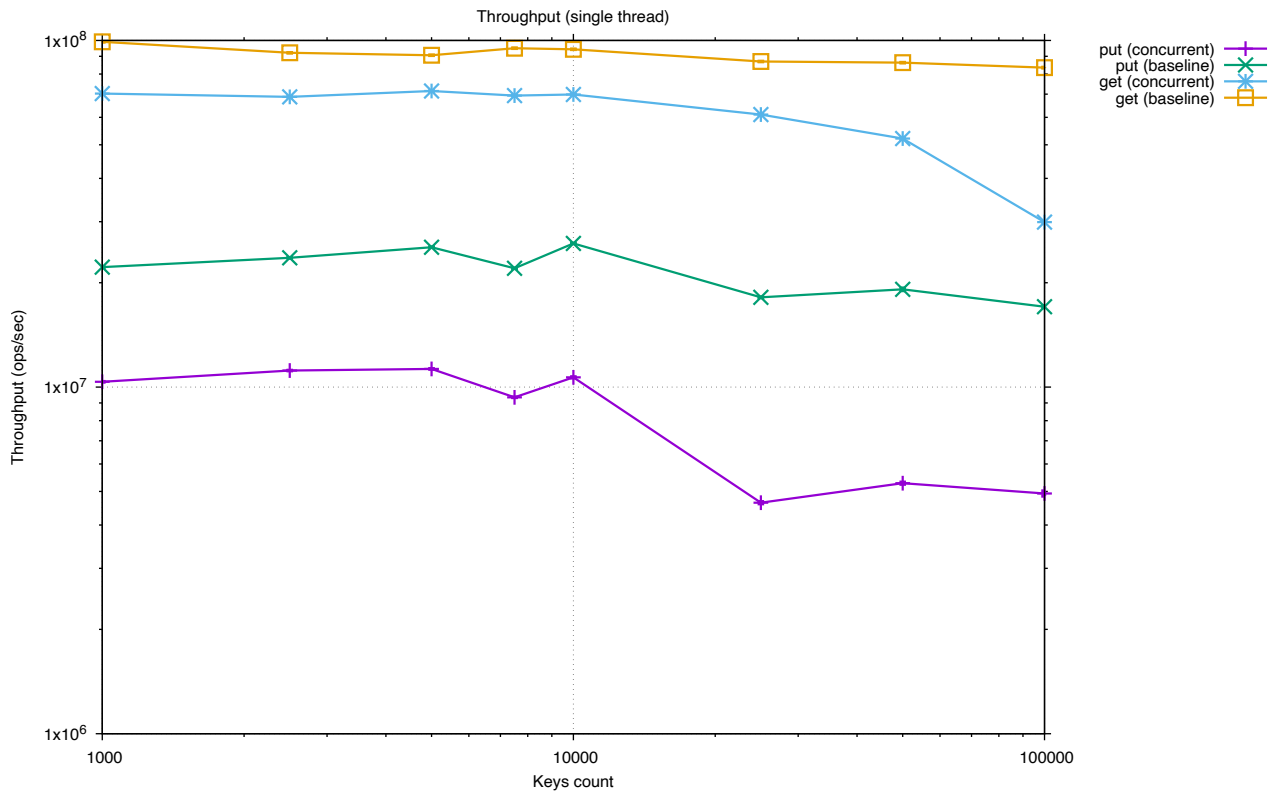


Рисунок 3.2 – Пропускная способность в однопоточном режиме

В однопоточном режиме реализация на основе `sync.RWMutex` ожидаемо медленнее `map[string]string` примерно в 2–4 раза для `put` и в 1.4–2.8 раза для `get`, что объясняется накладными расходами на захват/освобождение мьютексов и атомарные операции. Задержка `put` растёт с увеличением N из-за ресайзов и увеличения длины цепочек в бакетах, тогда как `get` остаётся стабильным до $N \approx 25000$, после чего начинает расти из-за ухудшения локальности кэша.

3.4 Результаты для нескольких потоков

Таблица 3.3 содержит результаты замеров задержки в многопоточном режиме.

Таблица 3.3 – Задержка операций в многопоточном режиме (нс/оп)

N	вставка (concurrent)	вставка (syncmap)	поиск (concurrent)	поиск (syncmap)
1000	144.76 ± 0.22	65.75 ± 0.12	38.63 ± 0.06	37.17 ± 0.05
2500	151.08 ± 0.22	61.23 ± 0.15	42.95 ± 0.15	41.02 ± 0.15
5000	162.02 ± 0.40	61.41 ± 0.10	48.39 ± 0.22	44.66 ± 0.19
7500	191.48 ± 0.59	62.33 ± 0.10	48.64 ± 0.27	47.84 ± 0.23
10000	180.94 ± 0.59	62.97 ± 0.56	54.01 ± 0.22	51.95 ± 0.12
25000	286.90 ± 2.25	80.41 ± 0.28	57.45 ± 0.10	51.04 ± 0.09
50000	305.69 ± 1.49	77.61 ± 0.51	58.24 ± 0.11	51.32 ± 0.13
100000	309.06 ± 1.91	83.11 ± 0.77	60.03 ± 0.10	53.83 ± 0.24

Таблица 3.4 – Пропускная способность в многопоточном режиме (млн оп/с)

N	вставка (concurrent)	вставка (syncmap)	поиск (concurrent)	поиск (syncmap)
1000	6.94 ± 0.01	15.45 ± 0.03	26.44 ± 0.04	27.27 ± 0.03
2500	6.63 ± 0.01	16.52 ± 0.04	24.07 ± 0.09	25.27 ± 0.10
5000	6.19 ± 0.02	16.33 ± 0.03	21.30 ± 0.12	23.06 ± 0.12
7500	5.23 ± 0.02	16.07 ± 0.03	21.21 ± 0.15	21.22 ± 0.10
10000	5.54 ± 0.02	16.00 ± 0.04	18.74 ± 0.10	19.33 ± 0.06
25000	3.49 ± 0.03	12.46 ± 0.04	17.42 ± 0.03	19.61 ± 0.04
50000	3.27 ± 0.02	12.92 ± 0.08	17.18 ± 0.03	19.50 ± 0.05
100000	3.24 ± 0.02	12.07 ± 0.10	16.66 ± 0.03	18.60 ± 0.08

Графики результатов представлены на рисунках 3.3 и 3.4.

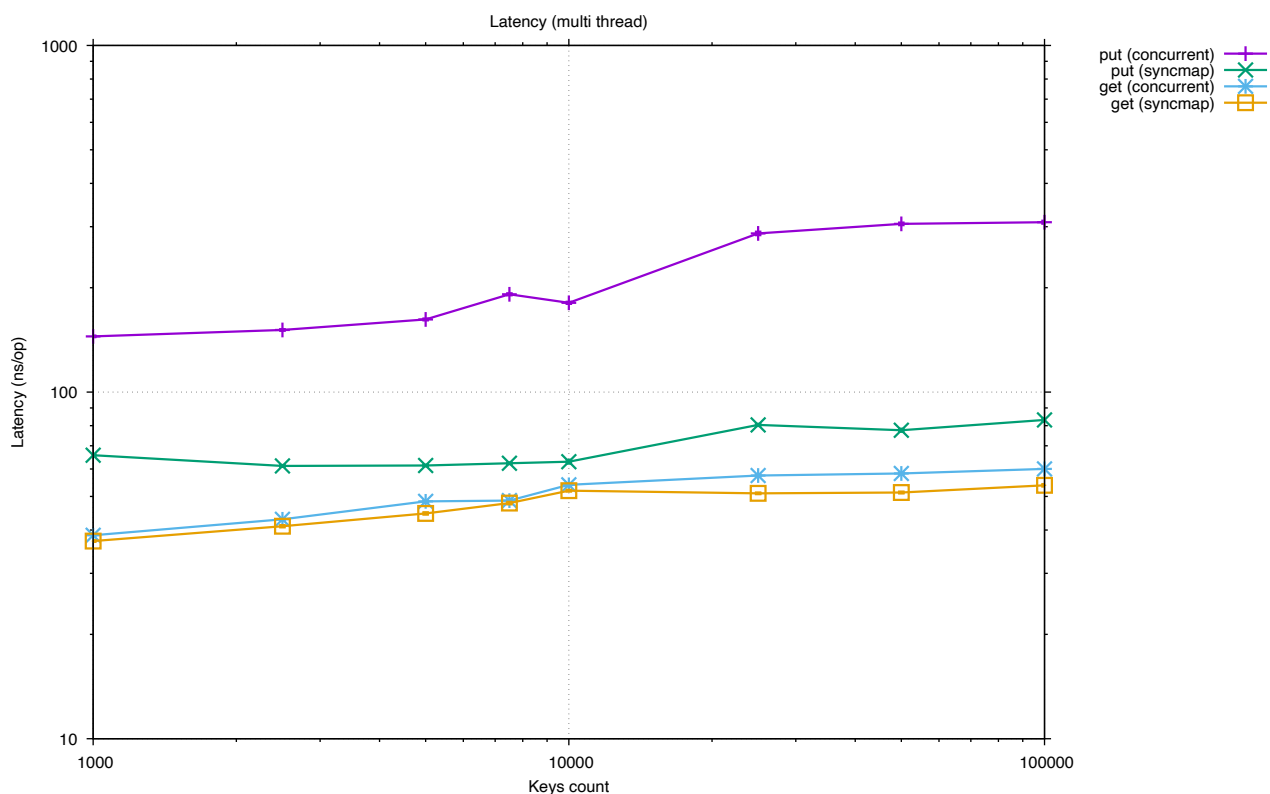


Рисунок 3.3 – Задержка операций в многопоточном режиме

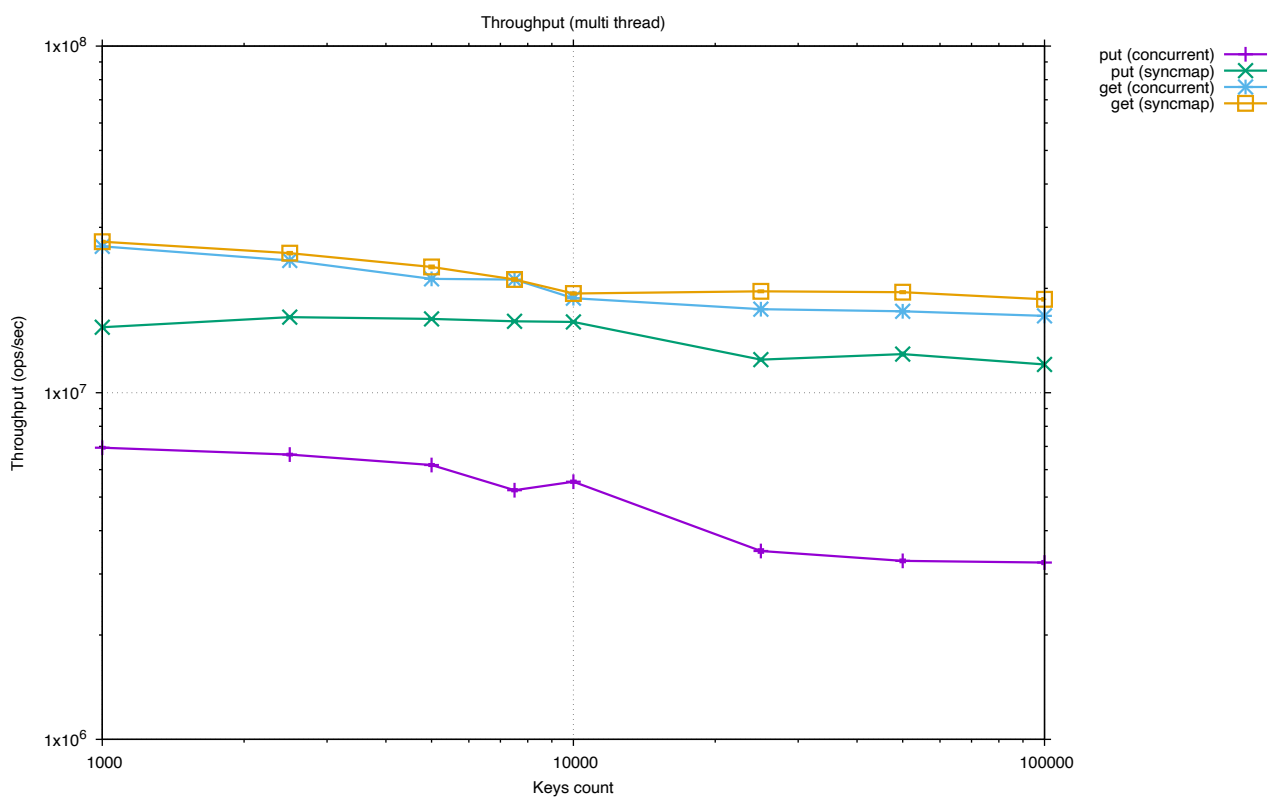


Рисунок 3.4 – Пропускная способность в многопоточном режиме

В многопоточном режиме `sync.Map` показывает лучшую пропускную

способность для **put** (в 2–4 раза), что объясняется оптимизацией **sync.Map** для сценариев с преобладанием чтения: записи накапливаются в dirty-карте и амортизированно переносятся в read-only кэш. Для **get** результаты сравнимы: на малых N разница минимальна, на больших — **sync.Map** имеет небольшое преимущество (10–15%).

Задержка **put** в реализации значительно растёт с увеличением N (от 145 до 309 нс/оп), что связано с конкуренцией за **resizeMu** при частых ресайзах и увеличением длины цепочек в бакетах. Задержка **get** растёт значительно медленнее (от 39 до 60 нс/оп), так как чтение не требует захвата **resizeMu** и использует **RLock** на бакете.

3.5 Потребление памяти

Таблица 3.5 содержит результаты замеров потребления кучи на операцию для **put** (для **get** потребление кучи составляет 0 байт/оп, так как временные аллокации успевают собрать GC между замерами **HeapAlloc**).

Таблица 3.5 – Потребление кучи на операцию вставки (байт/оп)

N	один поток (concurrent)	один поток (baseline)	много потоков (concurrent)	много потоков (syncmap)
1000	311.6	160.3	313.3	138.3
2500	253.3	129.7	254.1	133.8
5000	250.9	130.5	251.3	140.3
7500	324.1	167.9	324.5	142.8
10000	255.2	130.8	255.5	142.2
25000	257.2	57.3	258.5	133.3
50000	260.9	84.1	260.4	135.2
100000	259.1	62.6	259.3	141.9

Графики потребления памяти представлены на рисунках 3.5 и 3.6.

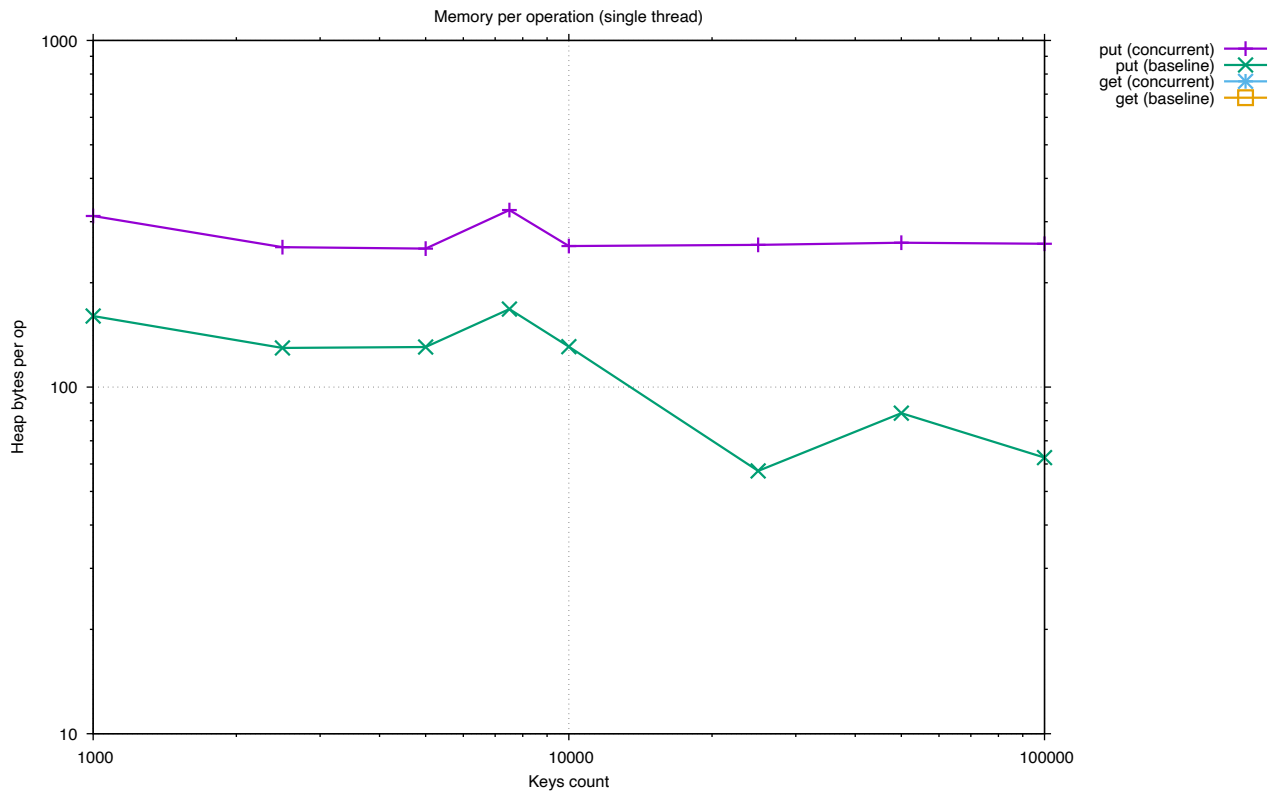


Рисунок 3.5 – Потребление памяти на операцию (однопоточный режим)

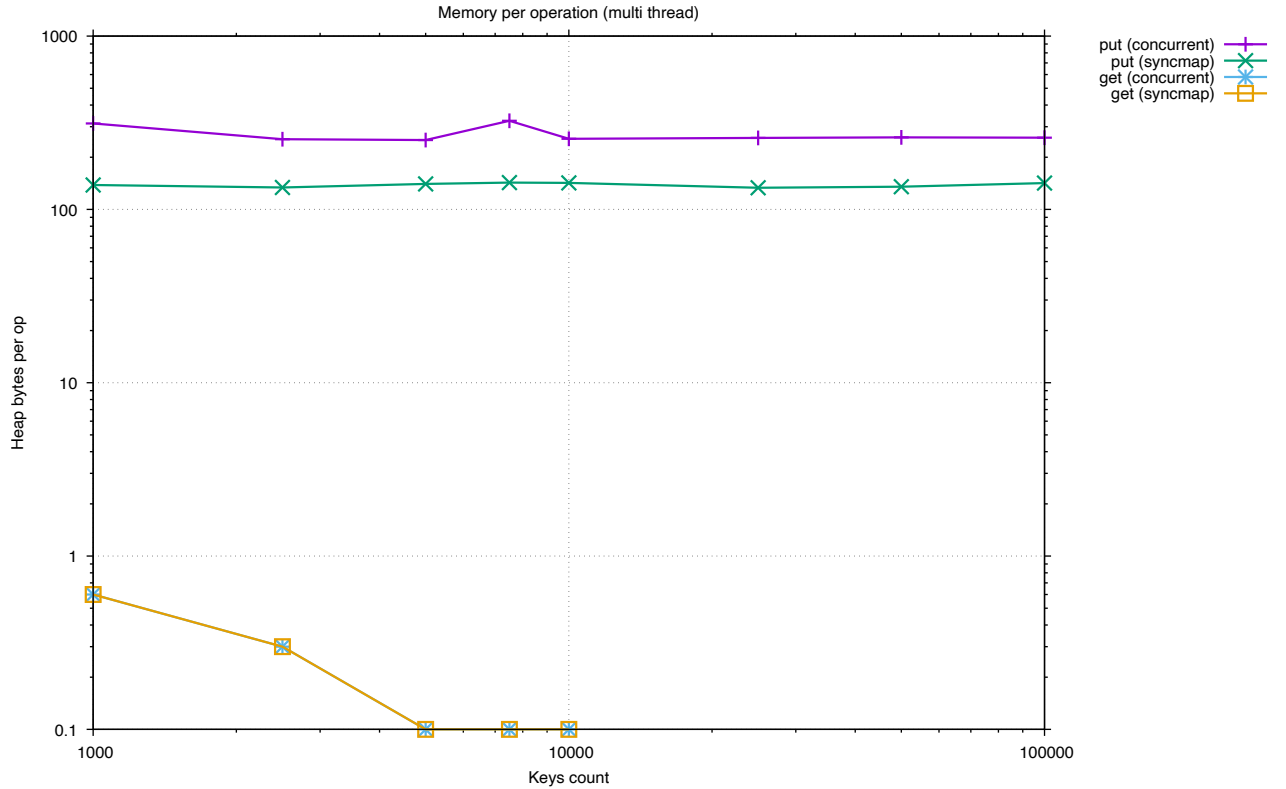


Рисунок 3.6 – Потребление памяти на операцию (многопоточный режим)

Потребление памяти на операцию `put` в нашей реализации стабильно

и составляет около 250–324 байт/оп, что примерно в 2 раза больше, чем у `map[string]string` (58–168 байт/оп) и `sync.Map` (133–142 байт/оп). Это объясняется оверхедом на `sync.RWMutex` в каждом бакете (24 байта), структурой `entry` с двумя строковыми заголовками (32 байта), а также дополнительными аллокациями при росте слайса `entries` в бакете. Значение 324 байт/оп при $N = 7500$ объясняется ресайзом таблицы (удвоение числа бакетов с 16 до 32 и перехэширование).

3.6 Профилирование

На рисунках 3.7 и 3.8 представлены графы вызовов CPU-профиля для операций `put` и `get` соответственно при $N = 50000$, 100 раундов, многопоточный режим.



4 Вывод

Реализована конкурентная хэш-таблица на основе striped locking с `sync.RWMutex` на каждый бакет, `atomic.Value` для lock-free доступа к таблице и динамическим ресайзом при превышении порога загрузки.

Проведены замеры производительности на наборах данных от 1000 до 100000 ключей в однопоточном и многопоточном режимах. Можно сделать следующие выводы:

- в однопоточном режиме реализация в 2–4 раза медленнее `map[string]string` для `put` и в 1.4–2.8 раза для `get` из-за накладных расходов на синхронизацию;
- в многопоточном режиме `sync.Map` показывает лучшую пропускную способность для `put` (в 2–4 раза), тогда как для `get` результаты сравнимы (отставание 10–15%);
- потребление памяти на операцию `put` стабильно и составляет около 250–324 байт/оп, что примерно в 2 раза больше, чем у `map[string]string` и `sync.Map`;
- профилирование подтверждает, что основное время тратится на захват мьютексов, вычисление хэша и сравнение ключей в бакетах.

Основным узким местом реализации является глобальная блокировка `resizeMu` при ресайзе, которая блокирует все операции `Put`. Для улучшения масштабируемости можно применить переносимые блокировки (lock striping с переносом) или lock-free схему ресайза.